



Modal

Serving Transformers

Lessons from the
Trenches of
Production Inference

Why inference?



swyx 
@swyx

Subscribe



everyone in ai infrastructure* is finally getting filthy rich and it is so nice to see them succeed

*not the sexy ai research stuff, just “boring” infra

12:26 PM · May 21, 2026 · **72.8K** Views

Why inference?

- Training is a *cost center*, inference is a *revenue center*
 - Will people pay for model weights per se? Unclear.
- Training is done by a few orgs, inference done by many
- Post-training now requires inference
- Inference straddles the stack right now – apps to algebra to electrons
- Incoming wave of inference demand (aka opportunity for engineers!)

Why listen to me?

- Working on inference at **Modal** for the last two years
 - And previously at FSDL, and on training before that (W&B, PhD @ Berkeley)
- Working internally on “GPU stuff” – metrics, profiling, perf
- Helping leading teams deploy+optimize inference on 100s/1000s of GPUs
- Eating my own dog food – Modal-hosted inference servers

FATHOM

hume

ramp

Cognition

runway

SUNO

blend

Doppel

substac

Quora

reducto

Lovable

David AI

Contextual AI

PostHog

Physical Intelligence

Resolve.ai

Applications

Workloads & SLOs

Models & Engines

Hardware

Deployment

Debugging & o11y

Performance Optimization

What's Next?

Serving Transformers

Applications

Workloads & SLOs

Models & Engines

Hardware

Deployment

Debugging & o11y

Performance Optimization

What's Next?

Serving Transformers

LLM Application Archetypes: Infra-Brained Edition

- Chatbot+
 - Prototype: ChatGPT, Claude Code
- Background Agent
 - Prototype: Devin, Resolve, Ramp Inspect, OpenClaw
- Data Processor
 - Prototype: Reducto, Fathom

Applications

Workloads & SLOs

Models & Engines

Hardware

Deployment

Debugging & o11y

Performance Optimization

What's Next?

Serving Transformers

Workloads & Service-Level Objectives

- Workload definitions and SLOs establish design space
- Hand-off point between stakeholders and engineers (you!)

Defining workloads by model, tokens per request, & SLO

THE LLM ENGINEER'S
20 26
ALMANAC

Advisor Executive Summary Benchmarking Workloads Quant Formats Token Timing Simulator

LLM Engine Advisor

 I'm feeling curious

I want to serve **Qwen 3 235B-A22B ft** 

with **any engine** 

Clients should receive **the first token**  in

under **1 second**  95% of the time

I expect on average

128 tokens in / 1024 tokens out 

I want to see **the lowest latency** 

configuration

Defining LLM workloads via metrics

- QPS: how many queries per second
 - Under control of users, tricky to forecast
 - Consider variability/seasonality
- TPQ: tokens per query, in/out aka prefill/decode
 - Under control of users AND model, but can be estimated
- Prefix re-use
 - Mostly under control of users
- Latency budget (TTFT and TPOT/ITL)
 - Defined by application, typically per user
- Reason per-replica! Then scale replicas to match total QPS

LLM App Archetypes: Infra-Brained Edition

- Chatbot+
 - Prototype: ChatGPT, Claude Code
 - Figure of merit: *interactivity*, aka tok/s/user
- Background Agent
 - Prototype: Devin, Resolve, Ramp Inspect, OpenClaw
 - Figure of merit: *TTLT*
- Data Processor
 - Prototype: Reducto, Fathom
 - Figure of merit: *MTokens per dollar*

Benchmarking one replica

- Min Latency: Run N queries one at a time, record $1 / \text{mean latency}$
- Max Throughput: Dump N queries at once, record $N / \text{total latency}$
- Sweep at client rates between these two endpoints

Benchmarking one replica



Min Latency



Max Throughput

Developing an intuition for metrics

THE LLM ENGINEER'S
2026
ALMANAC

Advisor Executive Summary Benchmarking Workloads Quant Formats **Token Timing Simulator**

Token Timing Simulator

Text

Prose (The Waste Land) ▼

Input tokens

10

Output tokens

1000



Qwen 3.5 9B - DFlash ON ×

April is the cruellest month, breeding
Lilacs out of the dead land, mixing
Memory and desire, stirring
Dull roots with spring rain.
Winter kept us warm, covering
Earth in forgetful snow, feeding

Qwen 3.5 9B - DFlash OFF ×

Speak, and my eyes failed, I was neither
Living nor dead, and I knew nothing,
Looking into the heart of light, the silence.
Oed und leer das Meer.
Madame Sosostris, famous clairvoyante,
Had a bad cold, nevertheless
Is known to be the wisest woman in Europe,

Applications

Workloads & SLOs

Models & Engines

Hardware

Deployment

Debugging & o11y

Performance Optimization

What's Next?

Serving Transformers

Different constraints lead to different model choices.

- **"Efficiency-bound"**: intelligence satisfactory, cost is primary driver
 - Open models dominate here
- **"Capability-bound"**: highest intelligence not enough, maybe never enough
 - Proprietary models dominate
 - But (RL) fine-tunes of open models catching up

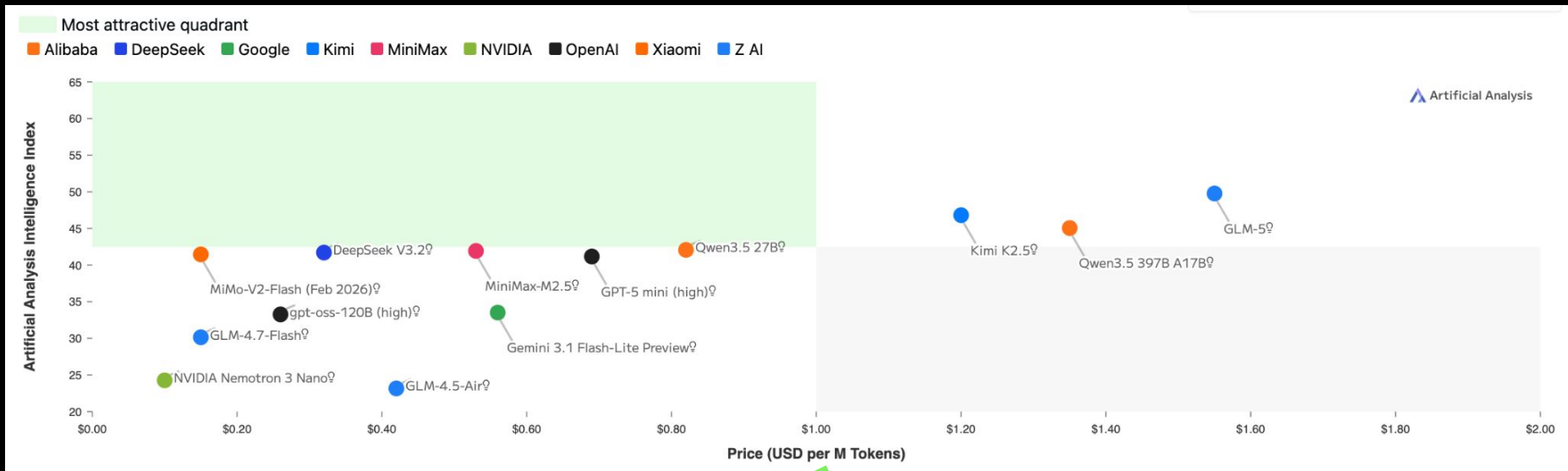
When capabilities satisfactory, cost is the primary driver.

- Efficiency-bound: model quality is “easy”, cost-perf is hard
- Typically single GPU per replica
 - Sometimes multiple GPUs (if low latency budget)
- Often data processor, *subagent* of bg agent, rarely chatbot+
- Often multimodal input, structured output
- Many options here:
 - Qwen, Nemotron, GPT-OSS, Gemma, Step, Mistral ...
 - Each of these is a model *family* (diff data mix, diff sizes, diff modalities)

When they are not, latency is the sticking point.

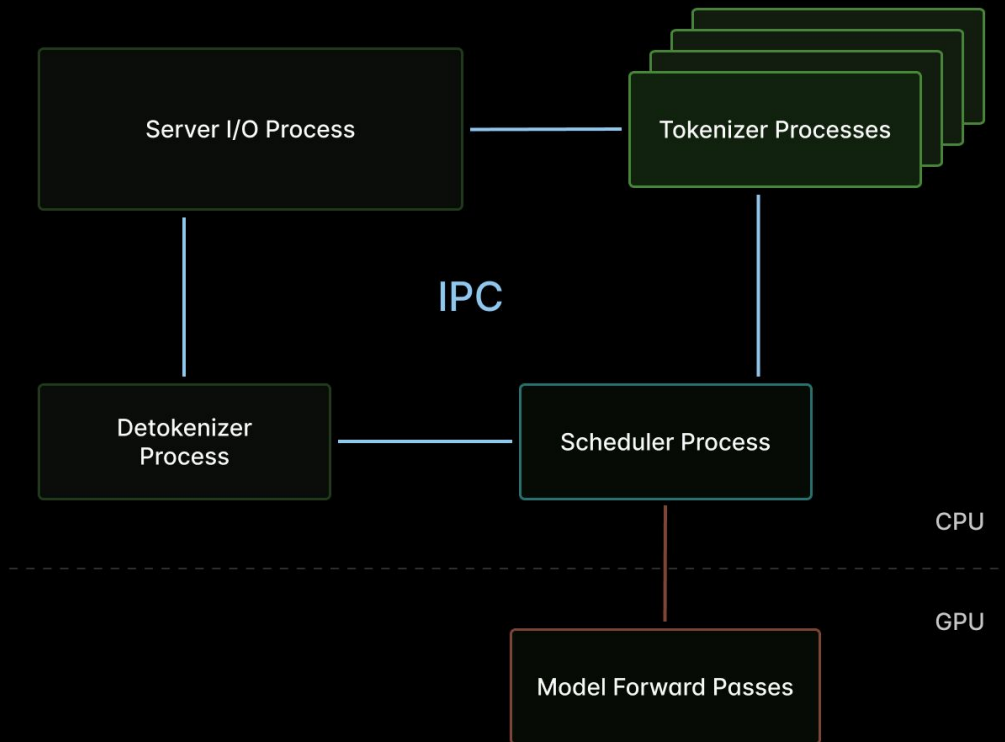
- Capability-bound: model quality is hard to hit
- Always multiple GPUs per replica
 - Sometimes multiple nodes per replica (if high latency budget)
- Often chatbot+ or "orchestrator" of bg agent, tool-call-heavy
- Often implemented with proprietary API first
- Fewer options here:
 - DeepSeek, Kimi, GLM, Minimax
 - Each a single model: largest/latest from provider

Tool: Artificial Analysis

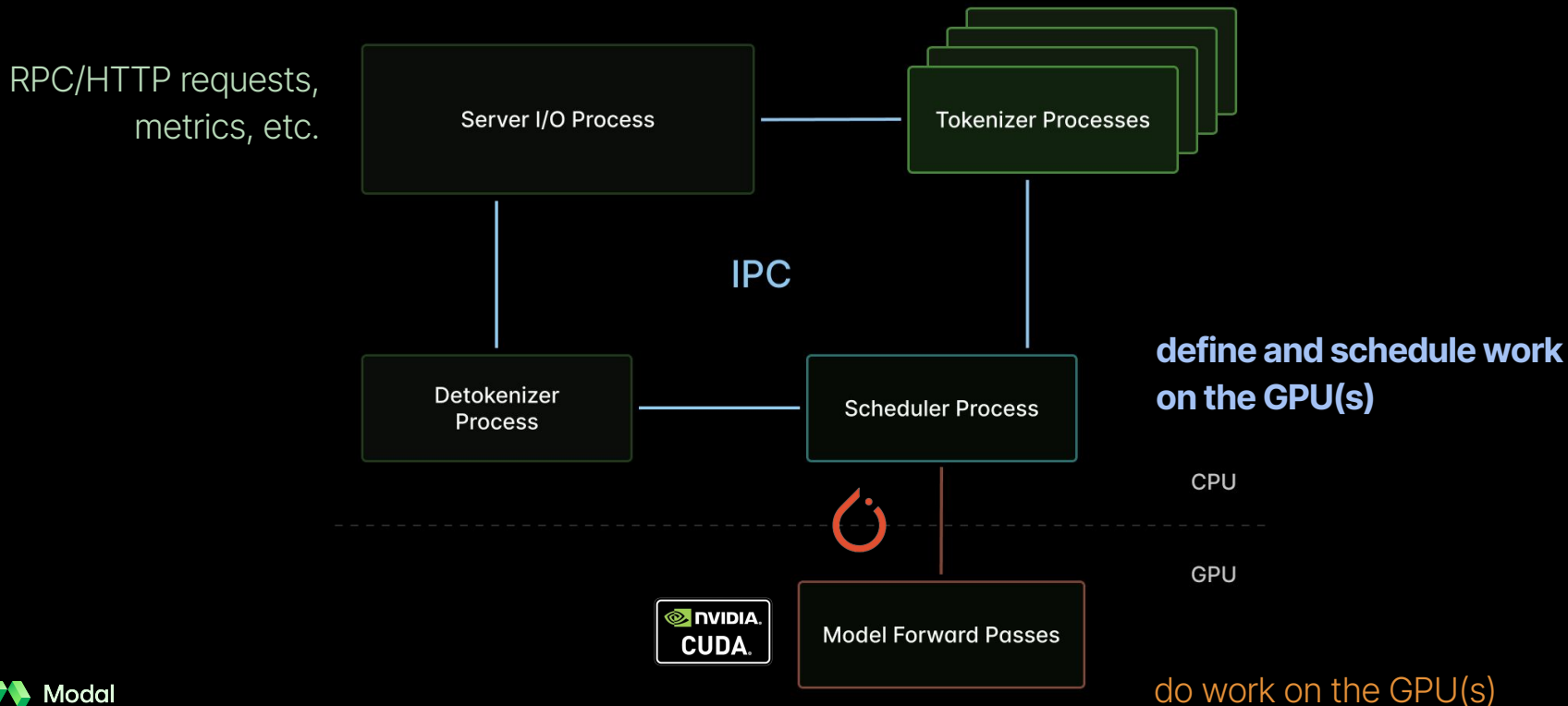


Based on inference providers.
Beating this is hard but not impossible!

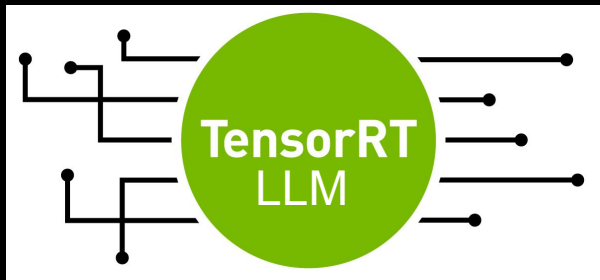
What is an "inference engine" really?



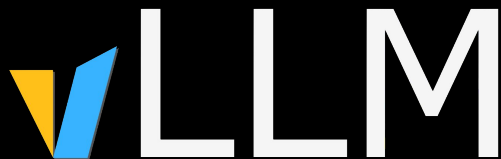
What is an “inference engine” really?



Choose your fighter



still best for small models,
small batches



first-to-market, "enterprise-y"



Gotta Go Fast™, "startup-y"

Choose your fighter

- TRT-LLM: still best for small models, small batch sizes
 - Compiled (Cpp) runtime – low host overhead, but painful development
 - NVIDIA still sometimes releases new techniques here
 - Decreased investment over time?
- vLLM: first-to-market, “enterprise-y”
 - Compatibility-oriented engineering culture
 - Major Deployers: LinkedIn, Meta, IBM/Red Hat, AWS, Anyscale
 - Open Governance: Linux Foundation > PyTorch Foundation > vLLM Project
 - Startup spinout: Inferact
- SGLang: Gotta Go Fast™, “startup-y”
 - Performance-obsessed engineering culture
 - Major Deployers: xAI, DeepSeek
 - Open Source, Closed Governance: LMSys Org non-profit
 - Startup spinout: RadixArk

Get to know your engines!

- `mini-sglang` and `nano-vllm` – simpler implementations of engine archs
- Aleksa Gordic's [walkthrough of vllm](#), Modal Notebooks [1], [2]
- DeepWiki from Cognition
 - Organized, diagrams, queryable
 - But frozen, nice to pair with your own local agent with repo access

Applications

Workloads & SLOs

Models & Engines

Hardware

Deployment

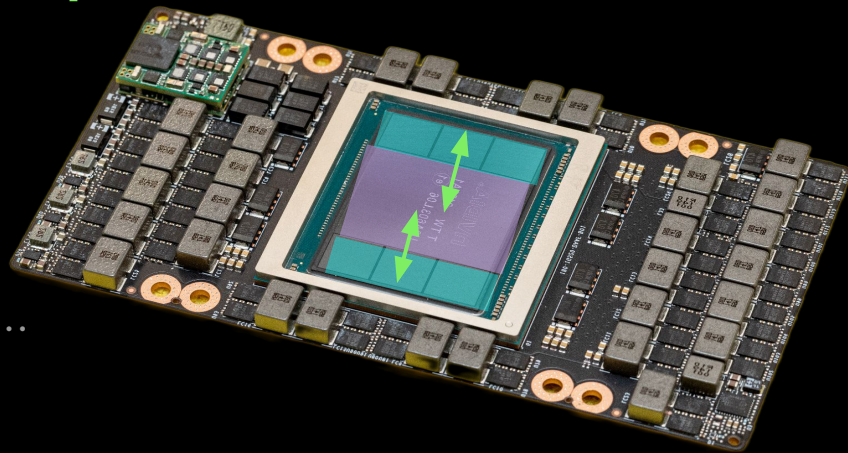
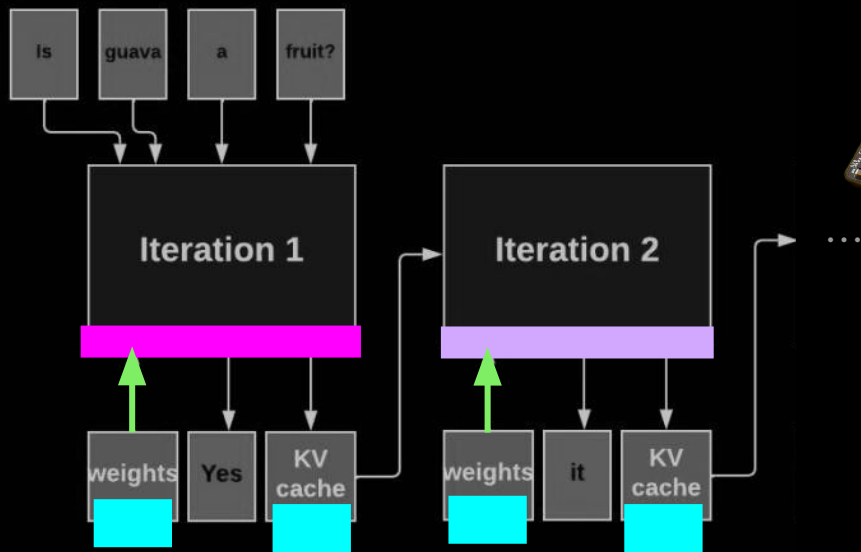
Debugging & o11y

Performance Optimization

What's Next?

Serving Transformers

Feels like two "sub-workloads": prefill and decode

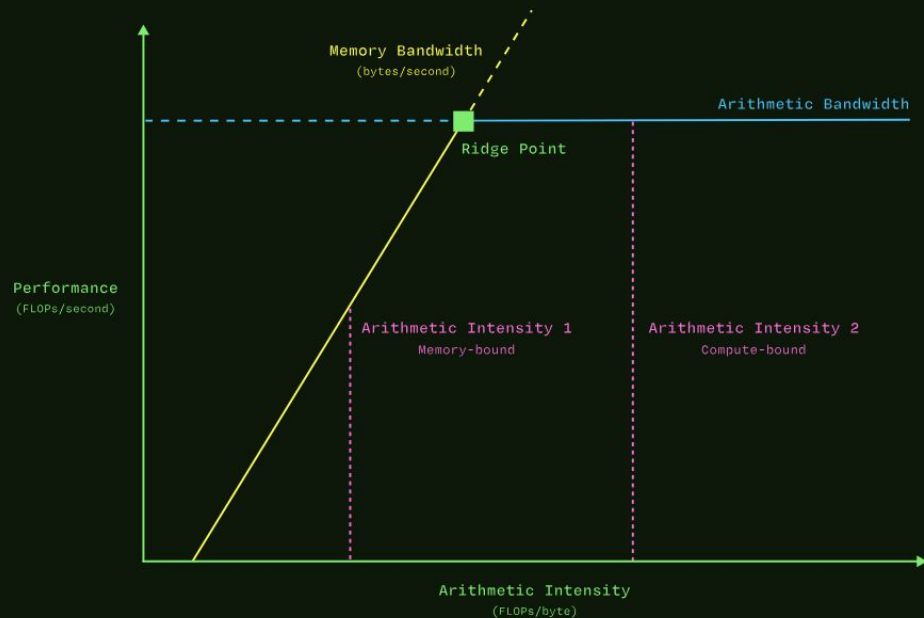


Move many GiB

Do many GFLOPs

Move many GiB

Do fewer GFLOPs

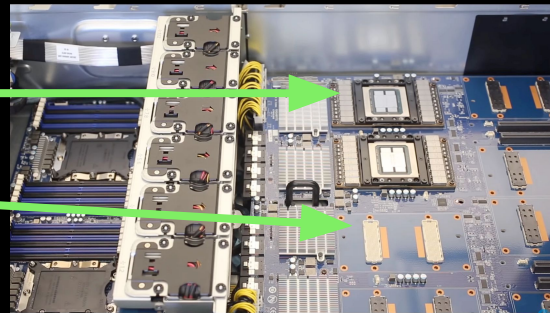


In the [roofline model](#), operational/arithmetic intensity is plotted on the horizontal axis. Diagram adapted from [Williams, Waterman, and Patterson \(2008\)](#).

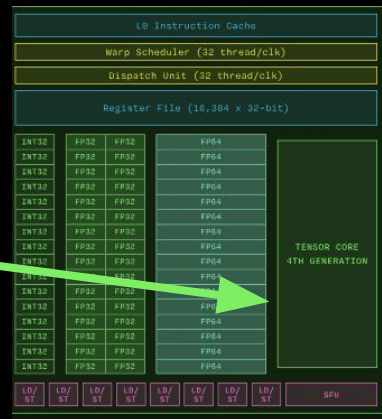
System (Compute / Memory)	Arithmetic Bandwidth (TFLOPs/s)	Memory Bandwidth (TB/s)	Ridge Point (FLOPs/byte)
A100 80GB SXM BF16 TC / HBM2e ↗	312	2	156
H100 SXM BF16 TC / HBM3 ↗	989	3.35	295
B200 BF16 TC / HBM3e ↗	2250	8	281
H100 SXM FP8 TC / HBM3 ↗	1979	3.35	592
B200 FP8 TC / HBM3e ↗	4500	8	562
B200 FP4 TC / HBM3e ↗	9000	8	1125

Use recent "Data Center" and "Tensor Core" NVIDIA GPUs.

- "Data Center": interconnect
 - HBM: high-bandwidth memory (not GDDR)
 - memory thrpt & capacity per GPU
 - SXM: mounting form factor (not PCIe)
 - full wattage, GPUs are *tightly* power-limited
 - NVLink/InfiniBand
 - Inter-GPU and inter-node connectivity



- "Tensor Core": matmul ASIC inside of GPU SM
 - ~99% of aggregate FLOP bw is here



Shameless Plugs: GPU Glossary and Tensara



tensara

Optimize Benchmark Repeat

A platform for GPU programming challenges. Write efficient GPU kernels and compare your solutions with other developers.

tensara.org

modal.com/gpu-glossary/readme



Modal

Other options?

- CPUs generally can't achieve acceptable latency, need HBM, lack programmer-controlled L1 cache ("smem")
- TPUs (currently) only have a single supplier e2e
- Other "inference chips" harder to program, generally single supplier
 - But watch this space!

Applications

Workloads & SLOs

Models & Engines

Hardware

Deployment

Debugging & o11y

Performance Optimization

What's Next?

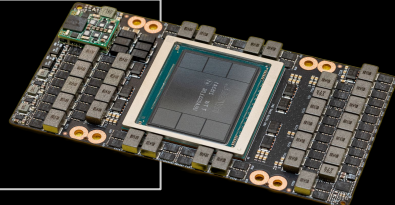
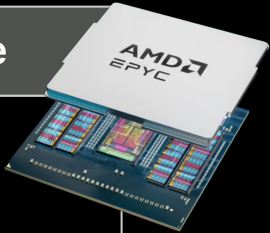
Serving Transformers

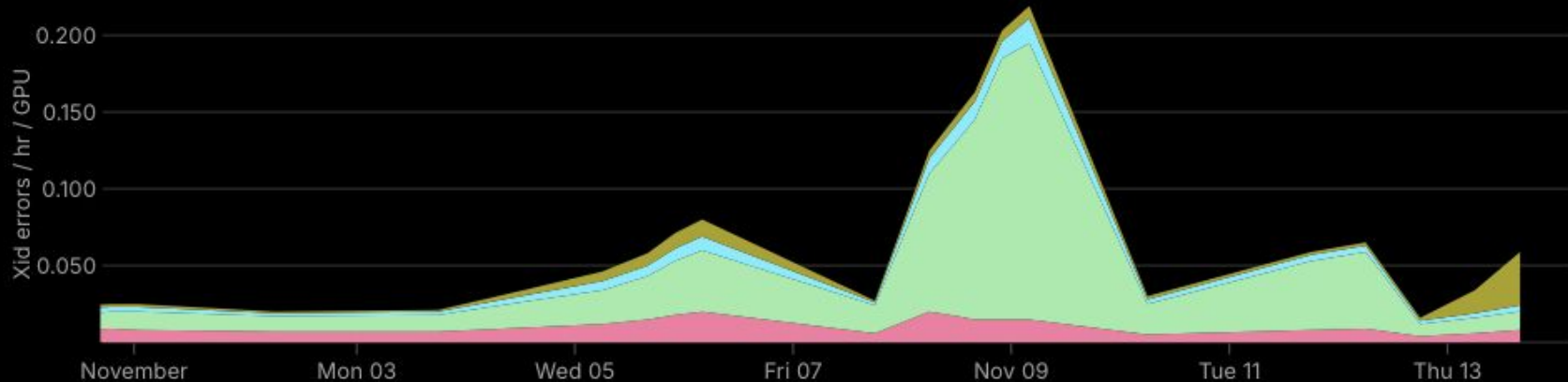
Deployment options are constrained by hardware

- GPUs and HBM are scarce, likely to remain scarce
- Inference mostly *served* right now, not local
 - Watch this space! Compare to databases (pgSQL vs SQLite)
- Chatbot+ latency budget eaten into by network
 - ~10ms regionally, ~100ms globally
- GPU Utilization should be maximized
 - Allocation Utilization
 - Kernel Utilization
 - SM Utilization
 - B/s & FLOP/s Utilization

GPUs fail a *lot*. Other hardware has spoiled us.

Component	Mean time to failure	Annual failure rate
CPU	~2,800 years	≈0.01%
Solid state drive (SSD)	~100 years	≈1%
NVIDIA H100	~50 <i>days</i>	≈700%





Critical level Xid errors per hour per GPU, grouped by (anonymized) cloud. Failure rates are far from negligible!

Engineering

December 28, 2025 • 8 minute read

Keeping 20,000 GPUs healthy



Jonathon Belotti @jonobelotti_IO
Member of Technical Staff

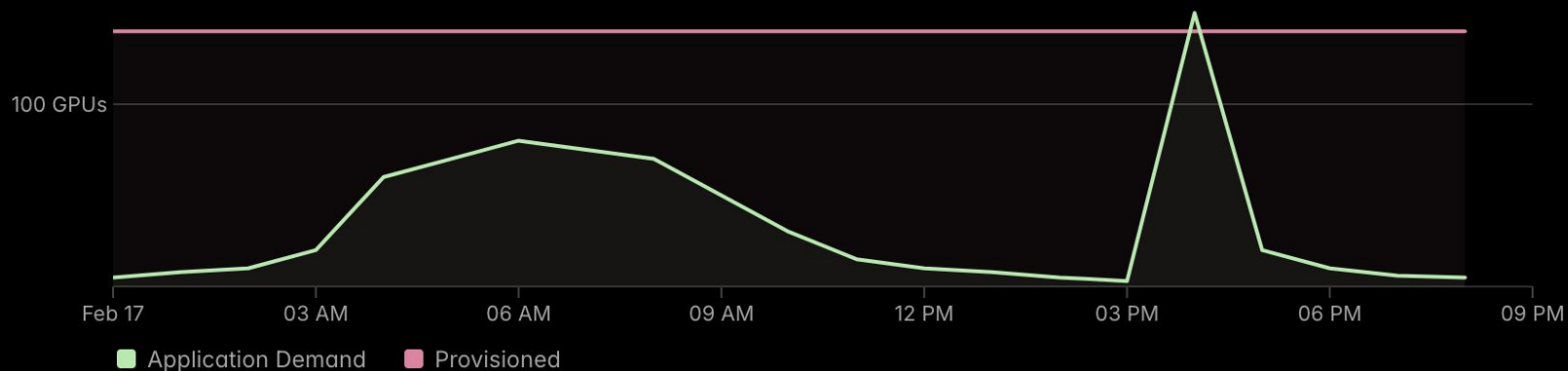


Amy Chang
Software Engineer Intern

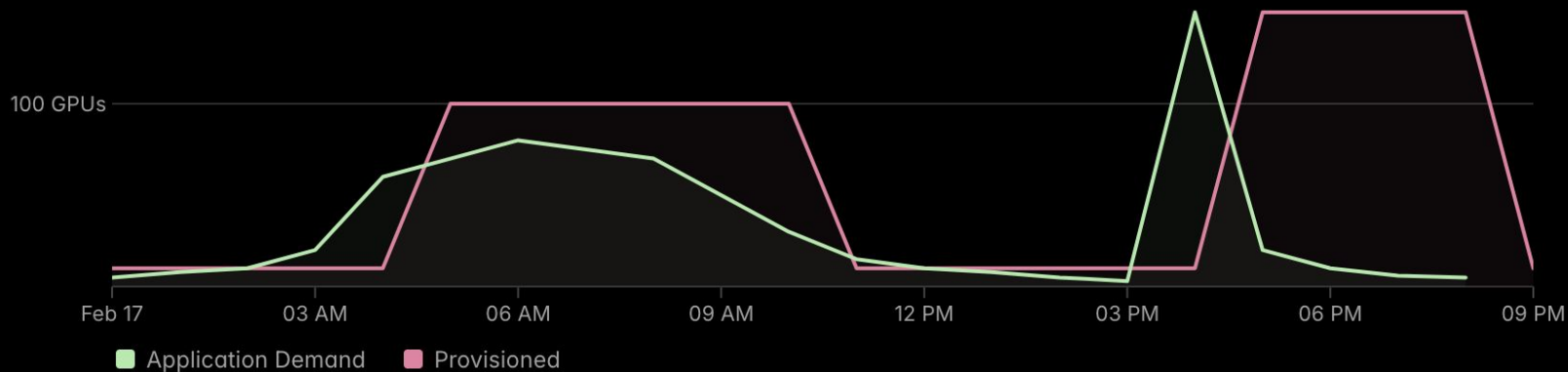
Traffic patterns for one month of inference serving (simulated)



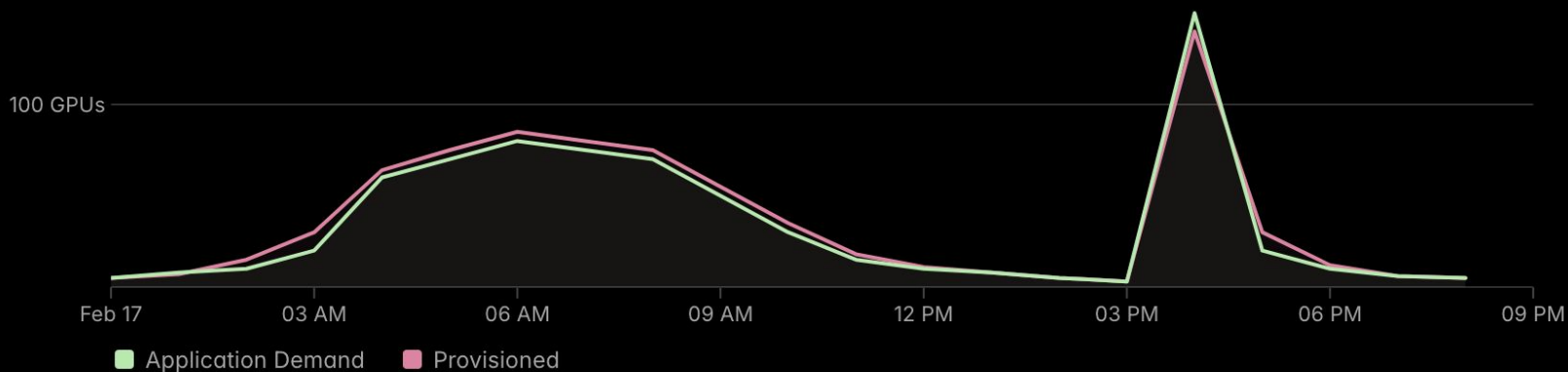
With a fixed, over-provisioned GPU allocation, utilization is low



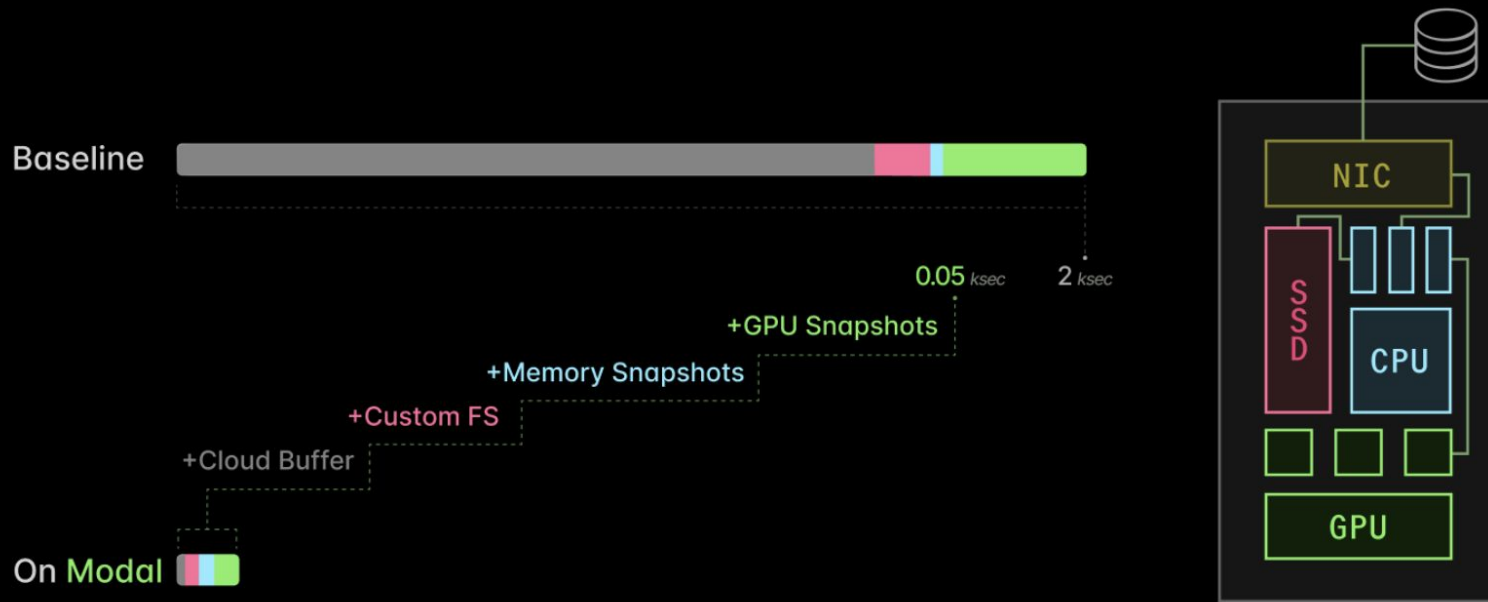
If allocation is slow, utilization and QoS suffer



With fast, automatic allocation, utilization and QoS can both be high



Together, these optimizations allow inference on Modal to spin up 40x faster: 50 seconds instead of 2k.



Machine Allocation: Cloud Buffer



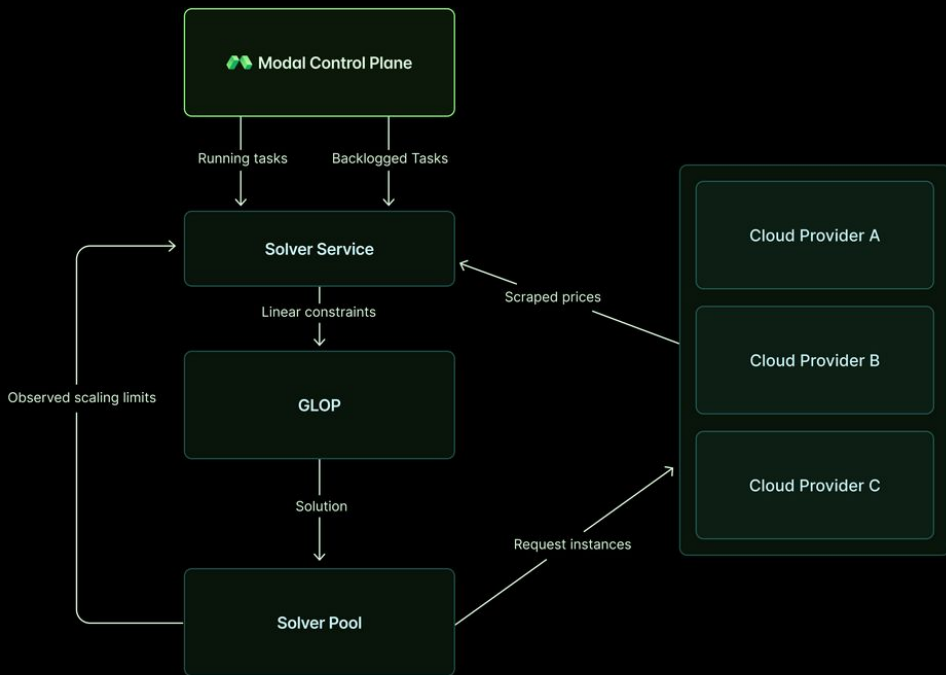
Machine Allocation: Cloud Buffer



Machine Allocation: Cloud Buffer



Machine Allocation: Cloud Buffer



Parameters:

A := total GPUs requested by users

B := count of GPUs in the buffer

$I = \{I_1, I_2, \dots, I_n\}$:= possible instance types

$\mathbf{C} = [C_1, \dots, C_n]$:= cost of instance types

$\mathbf{L} = [L_1, \dots, L_n]$:= scaling limits for each type

Output:

$\mathbf{X} = [x_1, \dots, x_n]$:= instances of each type to launch

Objective:

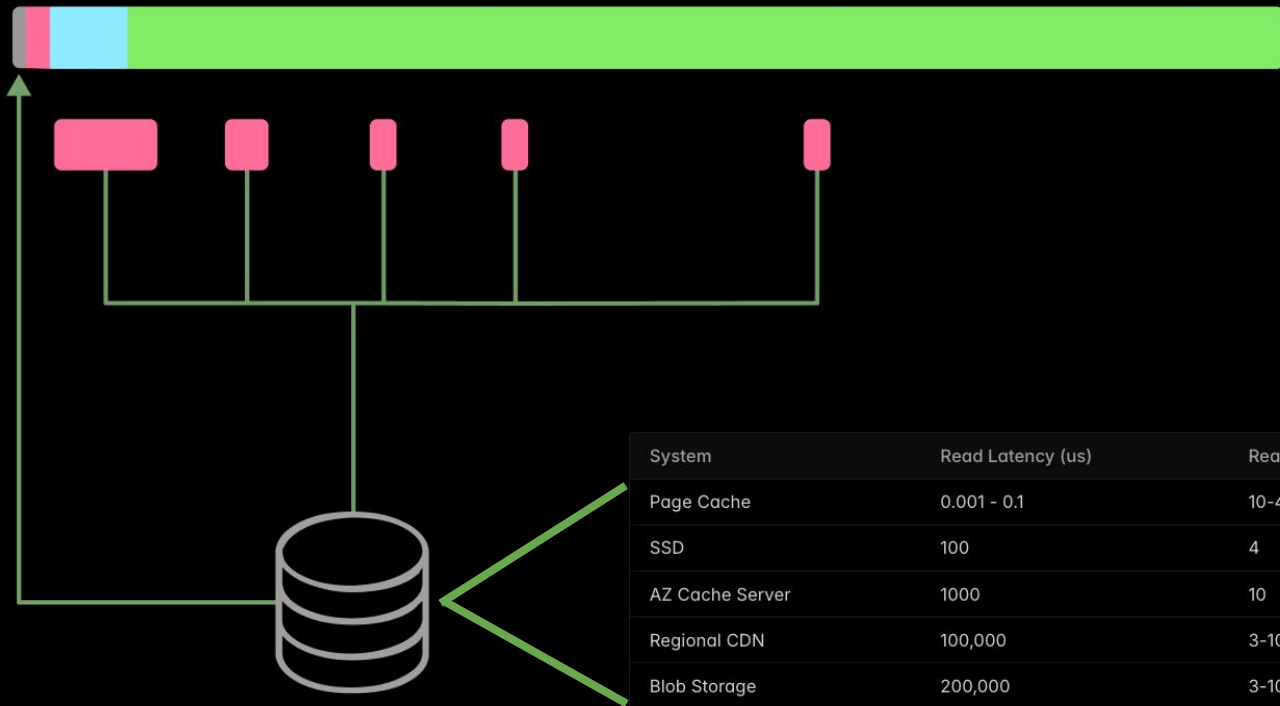
$$\text{minimize } \mathbf{C}^T \mathbf{X} = \sum_{i=1}^n x_i C_i$$

Constraints:

$$8 \sum_{i=1}^n x_i \geq A + B$$

$$x_i \leq L_i \quad (i = 1, \dots, n)$$

Container FS: Lazy + Eager + Content-Based Cloud Cache



System	Read Latency (us)	Read Throughput (GiB/s)
Page Cache	0.001 - 0.1	10-40
SSD	100	4
AZ Cache Server	1000	10
Regional CDN	100,000	3-10
Blob Storage	200,000	3-10

Application Start: Checkpoint/Restore

Memory Mappings



Threads

```
tid = 123
pid = 123
registers:
rip = 0x1234
rsp = 0x2345
rax = 0x9873
rdi = 0x2834
...
```

thread id
process id
instruction pointer
stack pointer
general purpose registers

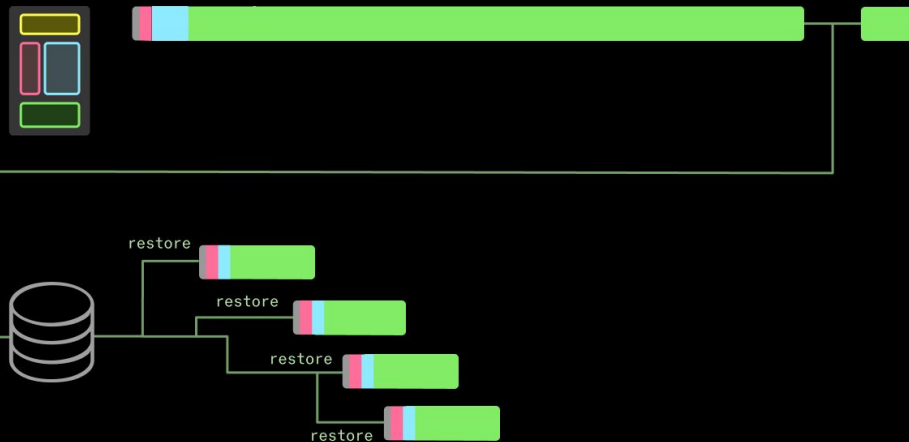
```
tid = 243
pid = 123
...
```

File Descriptor Table

```
0 = stdin
1 = stdout
2 = stderr
5 = myfile.txt
6 = <tcp:88>
7 = <tcp conn>
```

standard channels
(0, 1, 2)
open file
tcp listening socket
socket to tcp client

checkpoint



```

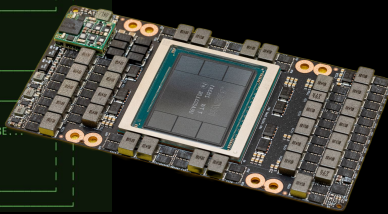
NODE 0: KERNEL
ID:      0 (topId: 1)
Kernel:  cutlass>x_sm100_simt_sgemm_f32_f32_f32_f32_
        64x32x16_1x1x1_3_tnn_align1_bias_f32_relu
        <<<[32,20],64,19200>>>
Node handle: 0x0000564604539520
Func handle: 0x0000564604AFC000

```

```

NODE 1: KERNEL
ID:      1 (topId: 0)
Kernel:  _ZN8cub::last8epilogue4impl12globalKernel11i8E...
        <<<[256,2],{8,32},0>>>
Node handle: 0x0000564604539C88
Func handle: 0x00005646044770F0

```



Applications

Workloads & SLOs

Models & Engines

Hardware

Deployment

Debugging & o11y

Performance Optimization

What's Next?

Serving Transformers

Some types of bugs you'll see

- Application-level bugs
 - Shared responsibility with application developers/stakeholders
- Model quality bugs
 - Tokenizer bugs
 - Unfortunately very common still, beaten down after model release
- Performance "bugs"
 - Regressions over time
 - Cross-replica differences

Observability: can you debug just from logs?

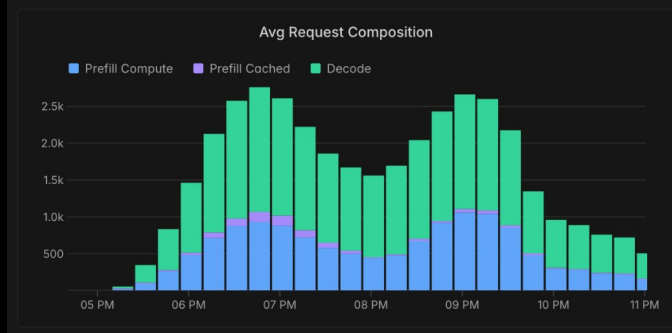
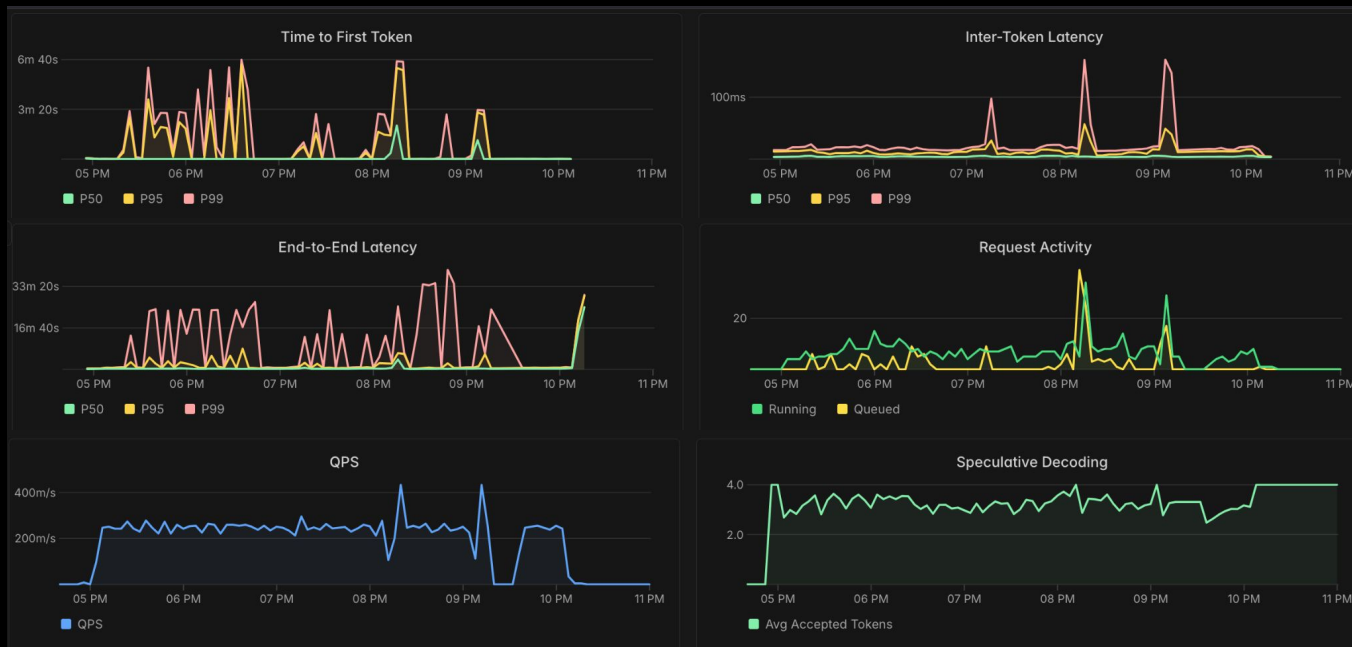
- Application-level: Bespoke.
 - Roll with OTel/Ddog or try a tool like LangSmith, Braintrust.
- Model quality: Run evals before deployment, log traces, log user feedback.
 - Tokenizers: Log *token IDs*.
- Performance: Log more metrics than you think you need.

Build evals to build observability for LLM apps.

- Like tests:
 - Thankless
 - Annoying, style-cramping
 - Extremely difficult and time-consuming
 - But critical to developing a successful application!
- Not just for debugging deployments
 - Make them model-agnostic and use them to drive application/workload-level decisions
 - Models and deployments are temporary. Evals are forever.
 - Use them for QA when fine-tuning, benchmarking, optimizing, training speculators...

Some metrics you should log

- TTFT, TPOT/ITL, TTLT
- RPS/QPS
- Queueing
- Prefill, cached prefill, decode
- Per replica, aggregate, p50/p95/p99/avg
- GPU temperature, power, kernel util, memory util



Applications

Workloads & SLOs

Models & Engines

Hardware

Deployment

Debugging & o11y

Performance Optimization

What's Next?

Serving Transformers

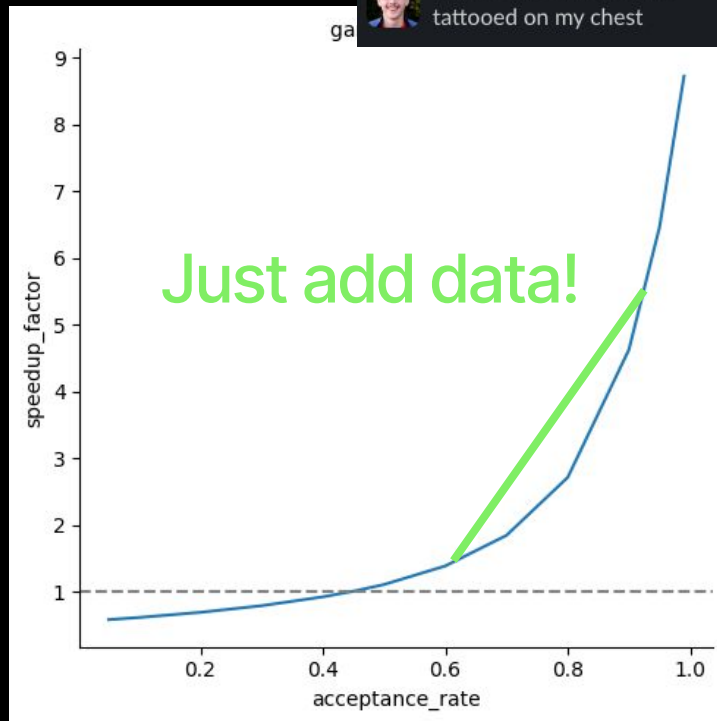
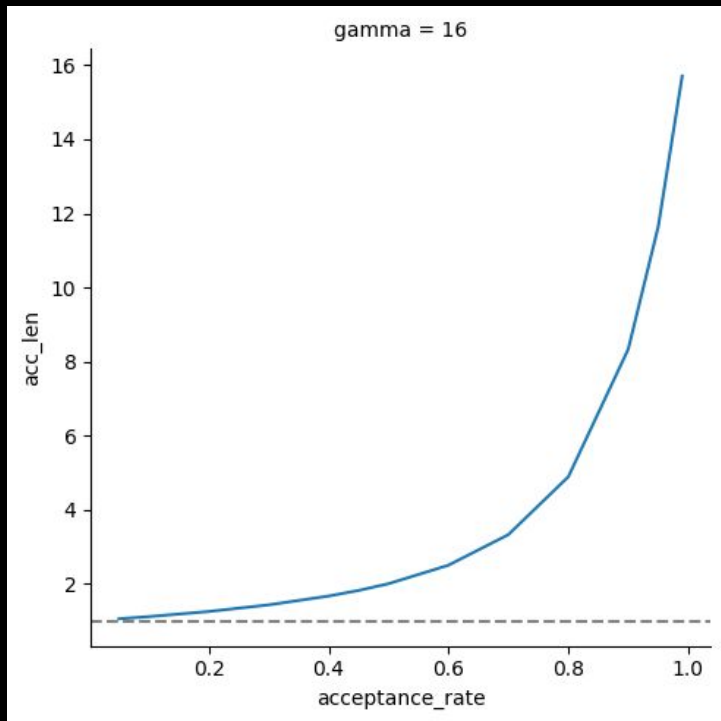
Performance Optimization

- Big things first
 - Speculative decoding – up to 6x faster
 - Quantization – ~2x faster
- Then a grind of medium tasks on host
 - Host overhead – typically up to 50%
- Then work on GPU-side
 - Kernel *optimization* – typically few percentage points

Speculation Is All You Need™



Speculation Is All You Need™



david.wang Tuesday at 5:47 PM

it's crazy how bitter lesson spec is 🦴

moar data moar compute brrr



charles Tuesday at 5:49 PM

tattooed on my chest

Many approaches to speculation.

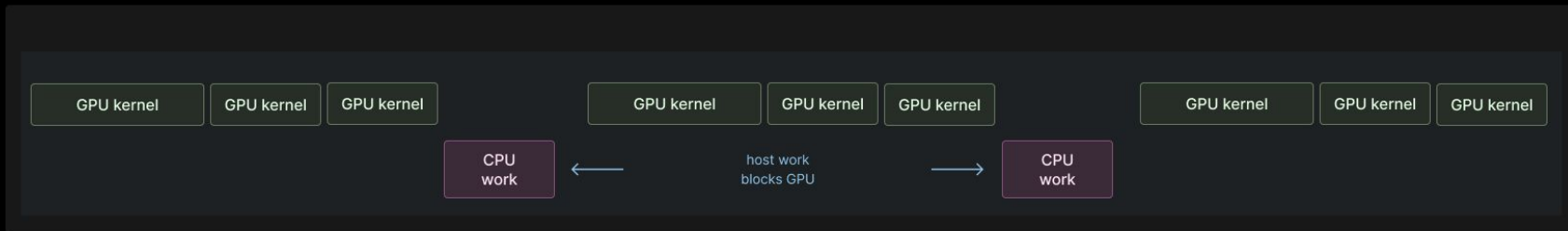
- N-gram: classical model of language, repeat what's in prompt
- MTP: trained into model (becoming standard: DSv3+4, Gemma 4, GLM 5, ...)
- EAGLE: added to model, based on hidden states
- DFlash: like EAGLE, but *de-noising* LM
 - We're betting on this class of approaches, have seen very strong results

Quantization rules, but requires full-stack coordination.

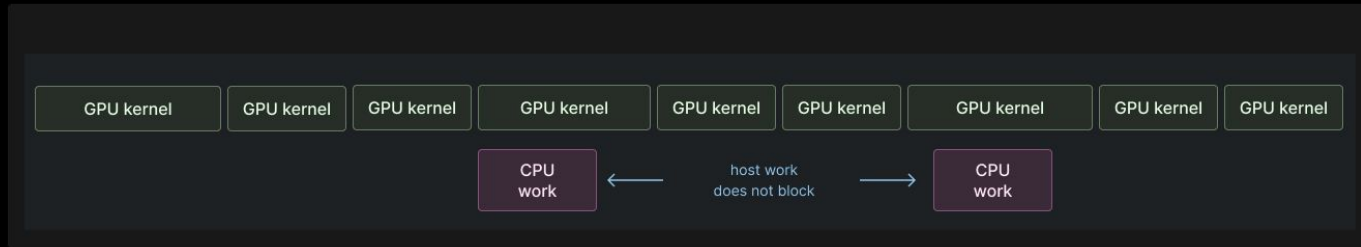
- Cuts memory bandwidth demand – fewer bytes
- Increases FLOP/s – linear scaling in TC b/w
- But this requires latest GPUs! Must check h/w layer
- Changes model behavior! Must check app layer
 - Worse on longer sequences (in or out)
 - BF16 mandatory, FP8 “standard”, FP4 cutting-edge
 - Attention harder to compress than matmuls (but matmuls are bottleneck)
 - KV cache quant/compression hard, should be done at train time

Host overhead: never block the GPU.

Before – host overhead



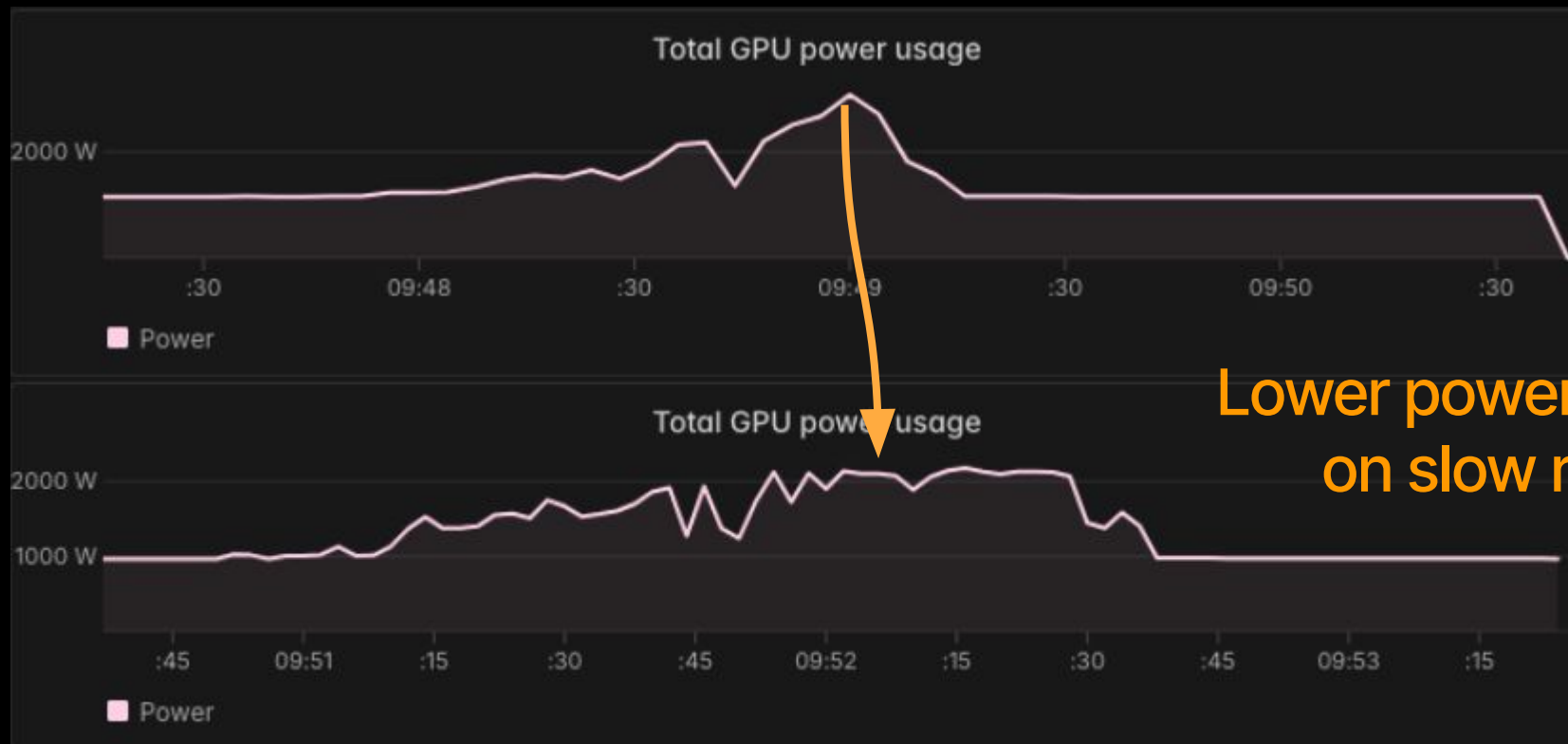
After – no host overhead



Technique: CUDA Graph capture

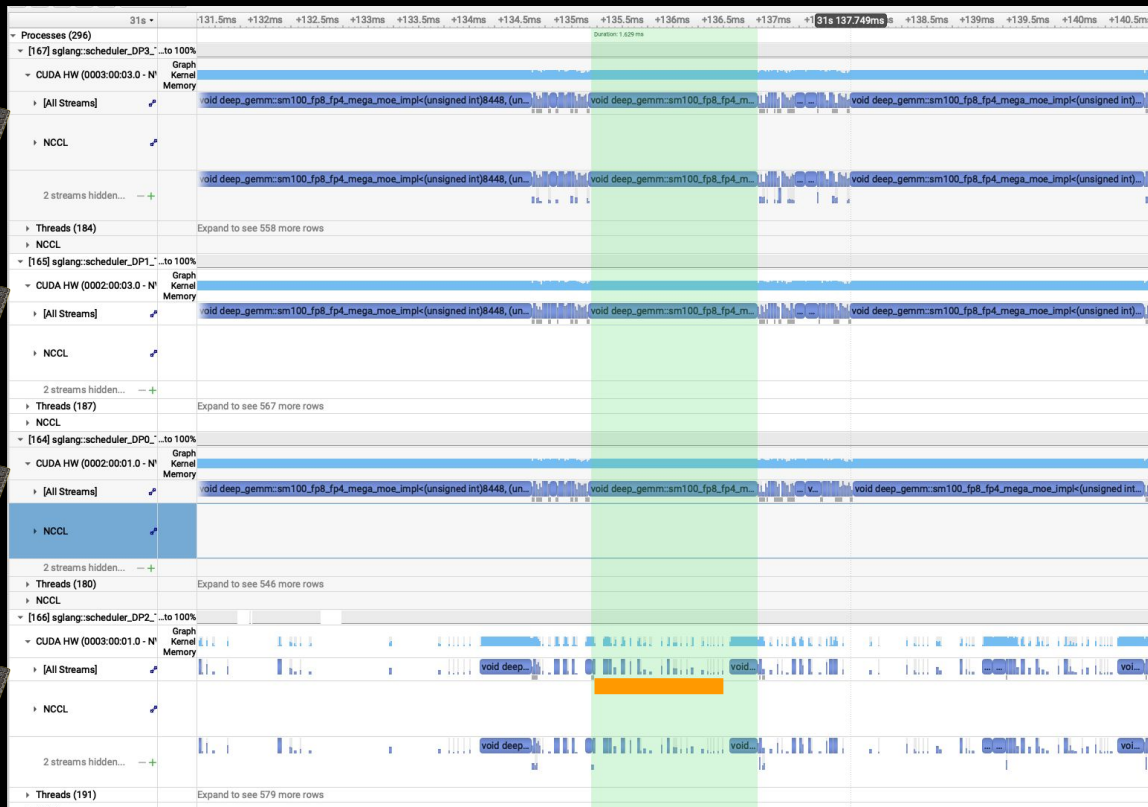
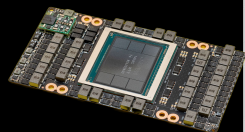
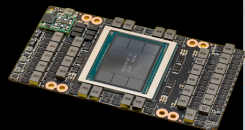
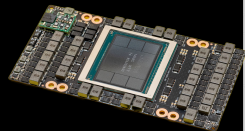
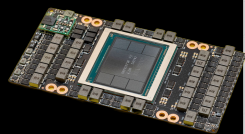


Tool: GPU Utilization and Temperature

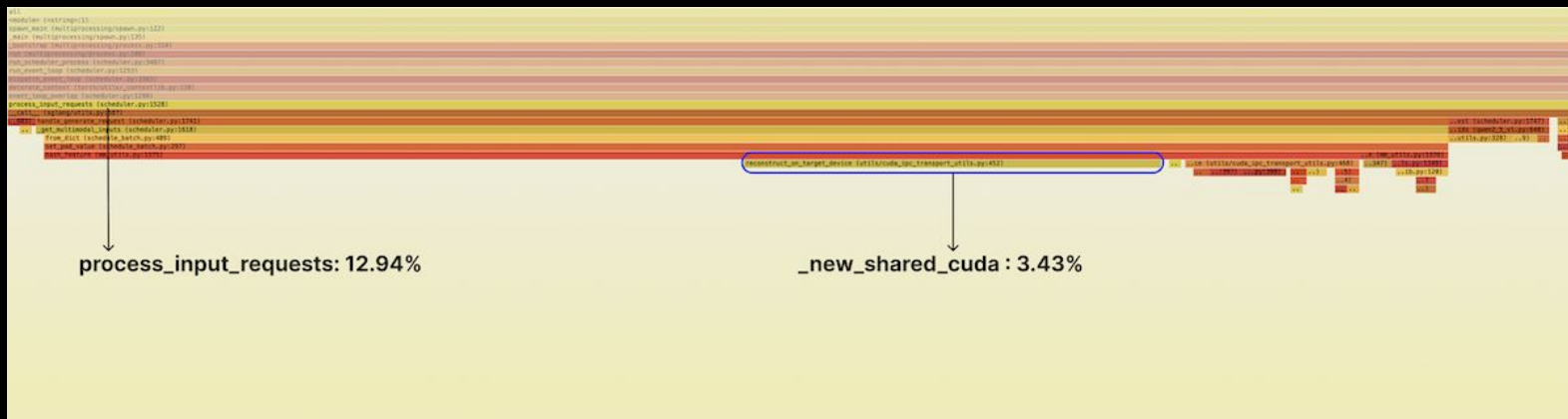


Lower power draw
on slow replica

Tool: NSight Systems/Torch Profiler (CUPTI)



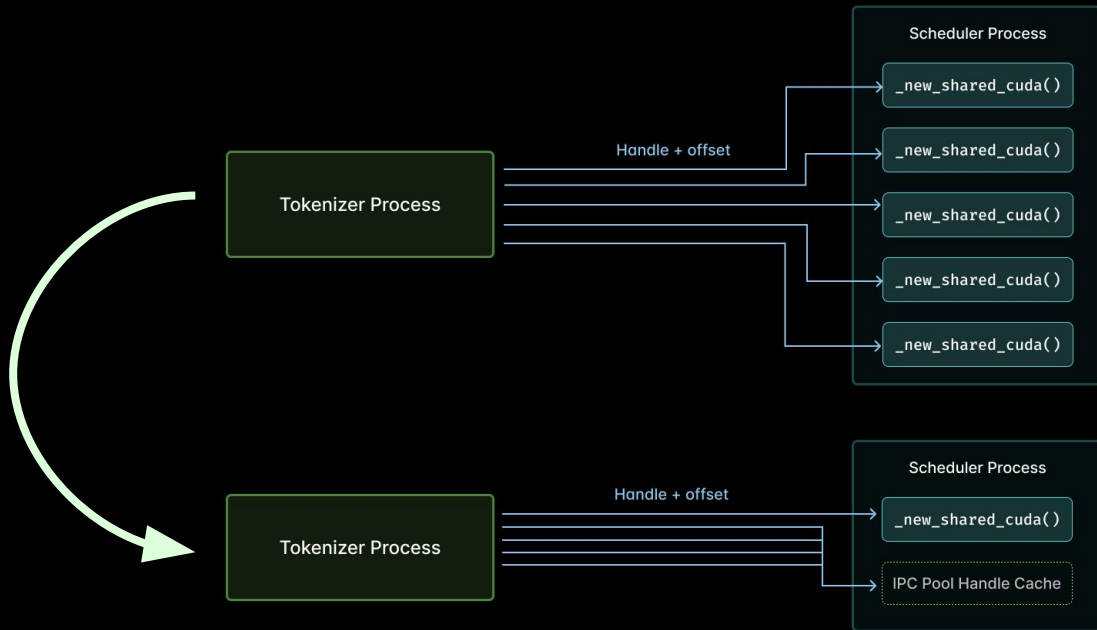
Tool: Literally just py-spy



modal.com/blog/boosting-multimodal-inference-performance-by-greater-than-10-with-a-single-python-dictionary

Tool: Literally just py-spy

>10% faster



Only then consider kernel optimization.

- Kernels are often already close to “speed-of-light”.
 - NVIDIA, DeepSeek, Tri Dao/Jay Shah, and others release these
 - Kernels are very reusable, kernels are released with models.
 - Good libs: CUBLAS, cudnn, CUTLASS, FlashInfer, DeepGEMM, FlashAttention, etc.
- There are only percentage points to gain there.
 - Still matters at scale! But don't optimize it *prematurely*.

Tool: NSight Compute

Measure hardware util

Welcome X profile.ncu.ncu-rep X

Result Size Time Cycles GPU SM Frequency Process Attributes

Current 542 - toyKernel (4, 1, 1)x(64, 1, 1) 488.16 us 644,328 0 - NVIDIA A10G 1.32 Ghz [67] a.out

Summary Details Source Context Comments Raw Session

Compare Tools View Export

This table shows all results in the report. Use the column headers to sort the results in this report. Double-click a result to see detailed metrics. Double-click on demangled names to rename it.

ID	Estimated Speedup [%]	Function Name	Demangled Name	Duration [us] (3614.37 us)	Runtime Improvement [us] (3433.65 us)	Compute Throughput [%]	Memory Throughput [%]	# Registers [register/thre]	Grid Size	Block Size [block]
0	95.00	toyKernel	toyKernel(int *)	488.16	463.75	0.00	0.02	16	4, 1, -	64, 1, -
1	95.00	toyKernel	toyKernel(int *)	279.71	265.73	0.00	0.05	16	4, 1, -	64, 1, -
2	95.00	toyKernel	toyKernel(int *)	772.61	733.98	0.00	0.02	16	4, 1, -	64, 1, -

Welcome X profile.ncu.ncu-rep X

Result Size Time Cycles GPU SM Frequency Process Attributes

Current 542 - toyKernel (4, 1, 1)x(64, 1, 1) 488.16 us 644,328 0 - NVIDIA A10G 1.32 Ghz [67] a.out

Summary Details Source Context Comments Raw Session

View: SASS

Source: toyKernel

Navigate By: Scoreboard Dependencies

# Address	Source	Scoreboard Dependencies	Live Registers	Instruction Category	Register Dependencies	Predicate Dependencies	Uniform Register Dependencies
1 0000154a 8d26c300	MOV R1, c[0x0][0x28]		1	Movement			
2 0000154a 8d26c310	S2R R2, SR_TID.X		2	Miscellaneous			
3 0000154a 8d26c320	MOV R8, 0x4		3	Movement			
4 0000154a 8d26c330	ULDC.64 UR4, c[0x0][0x118]		3	Uniform Data			
5 0000154a 8d26c340	S2R R3, SR_CTAD.X		4	Miscellaneous			
6 0000154a 8d26c350	IMAD R2, R3, c[0x0][0x0], R2		4	Integer			
7 0000154a 8d26c360	IMAD.WIDE R2, R2, R3, c[0x0][0x160]		4	Integer			
8 0000154a 8d26c370	LDG.E R0, [R2.64]		4	Load/Store			
9 0000154a 8d26c380	SHF.L.U32 R0, R0, 0x1, R2		5	Integer			
10 0000154a 8d26c390	STG.E [R2.64], R0		4	Load/Store			
11 0000154a 8d26c3a0	NANOSLEEP 0xf4240		1	Control			
12 0000154a 8d26c3b0	EXIT		1	Control			
13 0000154a 8d26c3c0	BRA 0x154a8d26c3c0		1	Control			
14 0000154a 8d26c3d0	NOP			Miscellaneous			
15 0000154a 8d26c3e0	NOP			Miscellaneous			
16 0000154a 8d26c3f0	NOP			Miscellaneous			
17 0000154a 8d26c400	NOP			Miscellaneous			
18 0000154a 8d26c410	NOP			Miscellaneous			
19 0000154a 8d26c420	NOP			Miscellaneous			
20 0000154a 8d26c430	NOP			Miscellaneous			
21 0000154a 8d26c440	NOP			Miscellaneous			
22 0000154a 8d26c450	NOP			Miscellaneous			
23 0000154a 8d26c460	NOP			Miscellaneous			
24 0000154a 8d26c470	NOP			Miscellaneous			

Map perf to code
(C++, PTX, SASS)

Tool: literally just a whiteboard

Feeds and Speeds

For $M=N=D=128$

Feeds on B200 (per SM):

1. Tensor Cores (BF16): $\frac{8192 \text{ ops}}{\text{cycle}}$
2. Exponential unit: $\frac{16 \text{ ops}}{\text{cycle}}$
3. Shared Memory traffic: $\frac{128 \text{ bytes}}{\text{cycle}}$

Speeds (clock-cycles per tile):

1. Forward (2 MMAs + MN exp)
 1. Tensor Cores: 1024
 2. Exp: 1024
 3. SMEM: 768
2. Backward (5 MMAs + MN exp): 1-CTA
 1. Tensor Cores: 2560
 2. Exp: 1024
 3. SMEM: 3328

Takeaway: Forward is bottlenecked by compute and exponential, backward is bottlenecked by shared memory bandwidth. So we overlap softmax with MMA in the forward pass and reduce shared memory traffic in the backward pass.

Applications

Workloads & SLOs

Models & Engines

Hardware

Deployment

Debugging & o11y

Performance Optimization

What's Next?

Serving Transformers

What's Next?

- “Unhinged” lossy optimizations
 - Move from shared providers to single-app deployments: *applications* define correctness
 - Lossy specdec, heavy quantization, layer skipping/pruning
- Megakernels
 - First for draft models, specific layers (e.g. DSv4 MegaMoE)
- Weirder hardware
 - NVIDIA VR LPX
 - ASICs (cf cryptocurrency blockchains)
 - Billion tok/s/user: analog computing? optical, memristive, ...

What's Next?

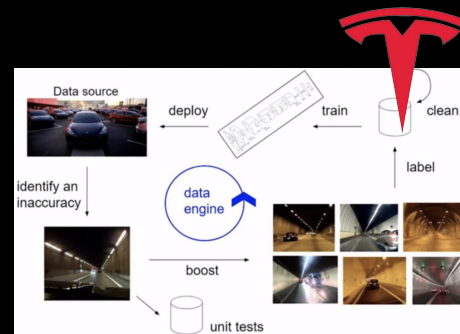
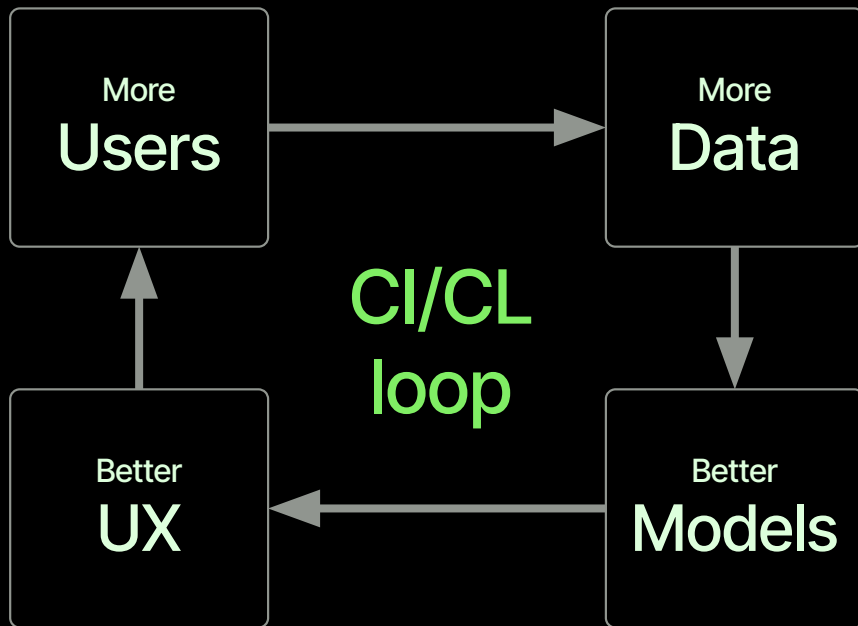
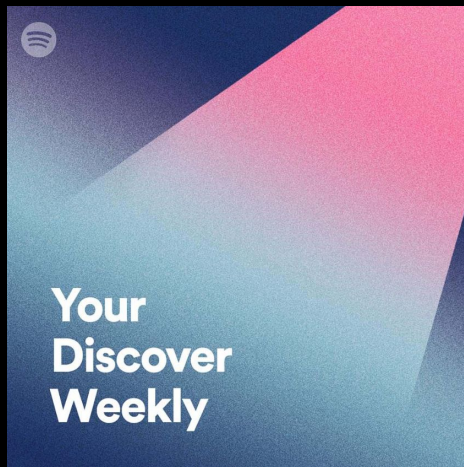


What's Next? Agentic software engineering.

- Now/Soon: Benchmarking, configuration, on-call.
 - Requires clear measures of correctness.
 - The agents don't take your job; they make your job *engineering the system that defines correctness*.
- Later: building bespoke engines per deployment?
 - [VibeServe](#), cf [llm.c](#) from Karpathy-san

What's Next? CI/CL

- Systems that automatically improve quality, robustness, and perf over time
 - E.g. automated speculator (re)training → automated distillation → continual learning



modal.jobs